Lab Report: Lab 6 - API Gateway Pattern and Access Control

Kyte

Contents

1. Cover Page
2. Abstract/Summary
3. Lab Specific Section: VI. API Gateway and Security 3.1 Objectives 3.2 Technology and Tools 3.3 Single Entry Point 3.4 Authentication Middleware 3.5 Cross Cutting Concerns
4. Conclusion & Reflection

1. Cover Page

Title: API Gateway Pattern and Access Control for the Kyte Project Management Tool

Course Info: Software Architecture

Student Details:

Họ và Tên	MSSV
Trần Thành Long	23010070
Nguyễn Xuân Mạnh	23010045

Date: Dec 16, 2025

2. Abstract/Summary

This lab describes how Kyte applies API gateway responsibilities in a single server entry point, centralizing routing and authentication before requests reach business logic.

3. Lab Specific Section: VI. API Gateway and Security

Note: Kyte is a modular monolith. Gateway composition is centralized in `packages/server/src/app.ts` and bootstrapped from `packages/server/src/index.ts`, rather than implemented as a separate service.

3.1 Objectives

- Centralize routing into one entry point.
- Enforce authentication before requests reach domain services.
- Simplify client integration with a single base URL.

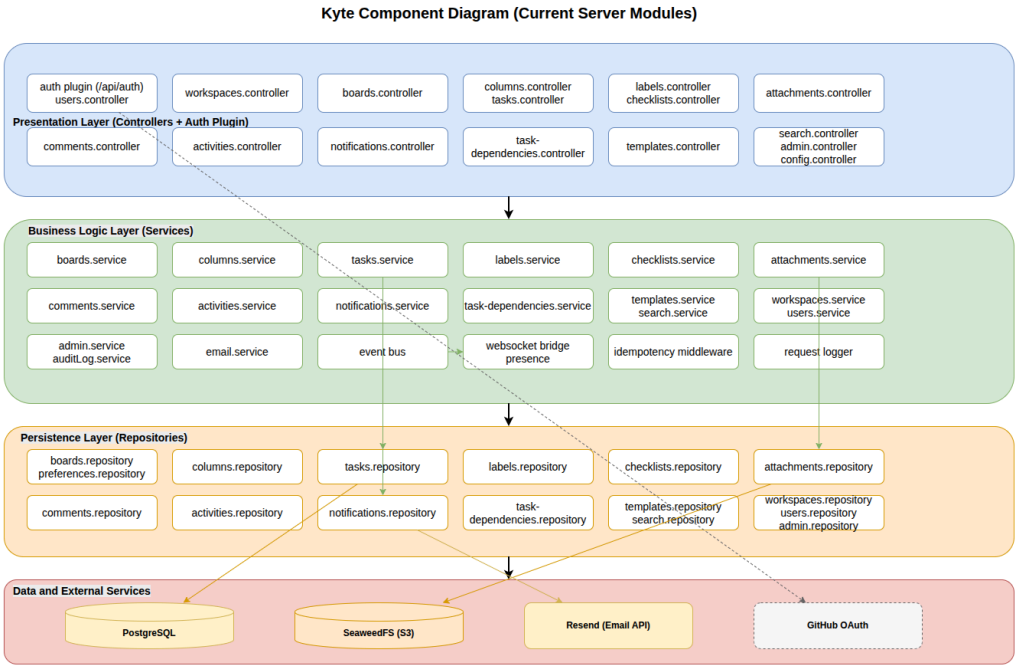
3.2 Technology and Tools

Tool	Purpose
Elysia	Core framework for routing and middleware.
Better Auth	Authentication middleware for session handling and identity context.
Eden Treaty	Type safe API integration for the web client.

3.3 Single Entry Point

The server entry point mounts all feature modules under a common API prefix:

```
const app = new Elysia()
  .use(requestLogger)
  .use(cors())
  .use(authPlugin)
  .group('/v1', (group) =>
    group
      .use(boardController)
      .use(taskController)
      .use(workspaceController)
      .use(adminController)
  )
  .listen(3000)
```



Lab 6 Component View

3.4 Authentication Middleware

- Incoming requests are validated by the auth middleware.
- If the session is invalid, the gateway returns `401` immediately.
- If valid, user context is attached and the request continues.

3.5 Cross Cutting Concerns

The gateway is the right place for consistent behavior across modules:

- Error formatting and status codes
- CORS policy enforcement
- Request logging for diagnostics

4. Conclusion & Reflection

Centralizing the gateway responsibilities improves security and simplifies client usage. It also provides a consistent place to enforce policies and add observability without duplicating code across modules.